

Chunking-Strategien für RAG am Beispiel des EU-GMP-Leitfadens

Tommi Jlassi

Fachhochschule Südwestfalen

Schriftliche Ausarbeitung im Modul „NLP“

bei Prof. Dr. Christian Gawron

30. September 2026

Inhaltsverzeichnis

1. Einleitung und Problemstellung	2
2. Grundlagen	3
2.1. Retrieval-Augmented Generation	3
2.2. Embeddings und Vektor-Retrieval	4
2.3. Chunking-Strategien	7
2.4. Stand der Technik	8
3. Methodik	9
3.1. Korpus	9
3.2. Chunking-Strategien im Vergleich	9
3.3. Fragenset und Ground Truth	10
3.4. Evaluation	11
4. Implementierung	12
5. Ergebnisse und Diskussion	14
5.1. Retrieval-Ergebnisse	14
5.2. Diskussion	16
6. Fazit und Ausblick	19

1. Einleitung und Problemstellung

In der pharmazeutischen Industrie ist die Arbeit stark durch Vorschriften geregelt. Der EU-GMP-Leitfaden legt fest, wie Arzneimittel hergestellt, geprüft und freigegeben werden müssen, und wer eine Frage zu diesen Regeln hat, muss nicht nur die passende Stelle im Leitfaden finden, sondern sich bei einer Entscheidung auch darauf berufen können. Genau hier kann Retrieval-Augmented Generation (RAG) helfen, da ein solches System nicht nur eine Antwort liefert, sondern auch die Textstellen, auf denen sie beruht [1], [2].

Damit das funktioniert, muss das System die richtige Stelle aber zuerst finden, und genau das ist bei normativen Dokumenten wie dem EU-GMP-Leitfaden besonders anspruchsvoll. Sie sind in Kapitel und nummerierte Absätze gegliedert, wobei jeder Absatz meist eine eigene, genau umrissene Anforderung enthält. Viele dieser Absätze behandeln dasselbe Thema und unterscheiden sich nur in Details wie Fristen oder Bedingungen. Hinzu kommt, dass Fachleute ihre Fragen oft mit anderen Begriffen stellen, als sie im Leitfaden stehen, etwa „media fill“ statt „aseptic process simulation“, und im deutschsprachigen Raum häufig auf Deutsch fragen, obwohl der hier verwendete Leitfaden auf Englisch vorliegt. Ein Fehler im Retrieval hat in diesem Umfeld größere Folgen als bei einer allgemeinen Wissensfrage, denn wird ein falscher oder unvollständiger Absatz gefunden, kann das zu einer Entscheidung führen, die nicht den Vorgaben entspricht.

Welche Stellen ein RAG-System überhaupt finden kann, hängt stark davon ab, wie die Dokumente vorher in Chunks zerlegt werden [3]. In vielen Systemen wird dafür einfach eine feste Länge gewählt, ohne auf den Aufbau des Dokuments zu achten. Qu et al. haben verschiedene Chunking-Verfahren verglichen und vermuten anhand einzelner Beispieldokumente, dass ein vorheriges Trennen entlang der Dokumentstruktur typische Fehler des größenbasierten Chunkings beheben würde [4, S. 11]. Experimentell geprüft haben sie diese Vermutung allerdings nicht. Außerdem besteht ein Teil ihrer Testdaten aus künstlich zusammengesetzten Dokumenten und nicht aus echten, stark gegliederten Texten [4, S. 3].

Ziel der Arbeit. Ziel ist es, diese Vermutung an einem echten Normdokument zu prüfen. Dafür wurde sich für einen Vergleich von zwei Strategien entschieden, einem größenbasierten Fixed-Size-Chunking und einem Struktur-Split, der den Text an den nummerierten Absätzen des EU-GMP-Leitfadens trennt. Alle anderen Teile des Systems, insbesondere das Embedding-Modell, bleiben dabei gleich, sodass Unterschiede allein auf das Chunking zurückgehen. Trainiert wird dabei nichts, da ein bereits vortrainiertes Embedding-Modell zum Einsatz kommt. Als Daten dienen stattdessen der EU-GMP-Leitfaden als Textkorpus und 55 Testfragen, deren richtige Antwort jeweils ein bestimmter Absatz ist. Die nummerierten Absätze dienen zugleich als Grundlage für die Bewertung,

denn da jede Frage einem bestimmten Absatz zugeordnet ist, lässt sich unabhängig von der Zerlegung genau prüfen, ob eine Strategie die richtige Stelle findet.

Die Arbeit beschränkt sich bewusst auf das Retrieval. Bewertet wird also, ob die richtigen Textstellen gefunden werden, und nicht, wie gut die Antworten sind, die ein Sprachmodell daraus formuliert.

Forschungsfrage. Welchen Einfluss hat die Chunking-Strategie auf die Retrieval-Qualität eines RAG-Systems für normative Dokumente am Beispiel des EU-GMP-Leitfadens?

Hypothese. *H1*: Strukturbasiertes Chunking erzielt einen höheren Recall als Fixed-Size-Chunking, weil zusammengehörige Anforderungen weder über Chunk-Grenzen zerrissen noch mit benachbarten Abschnitten vermischt werden. *H0*: Kein signifikanter Unterschied zwischen den beiden Strategien.

Aufbau der Arbeit. Kapitel 2 erklärt die Grundlagen von RAG, Embeddings und Chunking. In Kapitel 3 werden Korpus, Chunking-Strategien, Fragenset und Bewertung beschrieben, Kapitel 4 zeigt die technische Umsetzung. Kapitel 5 stellt die Ergebnisse vor und diskutiert sie, bevor Kapitel 6 die Arbeit mit einem Fazit und Ausblick abschließt.

2. Grundlagen

Sprachmodelle werden inzwischen von einer breiten Öffentlichkeit genutzt, und rund 50 % der Internetnutzerinnen und -nutzer greifen zumindest gelegentlich auf KI-Chats statt auf klassische Suchmaschinen zurück [5]. Ihre Antworten sind jedoch nicht immer korrekt, da Sprachmodelle dazu neigen, Inhalte zu halluzinieren, also Aussagen zu erzeugen, die zwar plausibel klingen, aber faktisch falsch oder nicht belegt sind [6]. Im beruflichen Alltag, in dem Entscheidungen auf belastbaren Fakten beruhen müssen, ist das ein ernstes Problem. RAG begegnet diesem Problem, indem es dem Sprachmodell zum Zeitpunkt der Anfrage passende Textstellen aus einer verlässlichen Wissensquelle bereitstellt [1]. Wie gut das gelingt, hängt maßgeblich davon ab, wie die zugrunde liegenden Dokumente in durchsuchbare Einheiten zerlegt werden [3]. Die folgenden Abschnitte erklären dazu die Grundlagen von RAG, Embeddings und Chunking.

2.1. Retrieval-Augmented Generation

Eine RAG-Pipeline lässt sich in vier aufeinanderfolgende Phasen gliedern [2]:

- Indexierung,
- Retrieval,
- Augmentierung,
- Generierung.

Bei der *Indexierung* wird die Wissensquelle vorbereitet. Die Dokumente werden in kleinere, möglichst in sich abgeschlossene Textabschnitte, sogenannte Chunks, zerlegt und jeder Chunk mithilfe eines Embedding-Modells in einen Vektor überführt und in einer Vektordatenbank abgelegt. Ohne eine solche Zerlegung müsste stets das gesamte Dokument als Kontext übergeben werden, sodass die benötigte Kontextlänge unmittelbar von der Dokumentgröße abhinge. Die Indexierung erfolgt vorab und wird nur selten wiederholt, etwa wenn neue Dokumente hinzugefügt oder bestehende aktualisiert werden. Sie bildet die durchsuchbare Grundlage des Systems.

Im *Retrieval* wird die Nutzerfrage mit demselben Embedding-Modell in einen Vektor überführt und anhand eines Ähnlichkeitsmaßes, typischerweise der Kosinus-Ähnlichkeit, mit den gespeicherten Chunk-Vektoren verglichen. Zurückgegeben werden die k ähnlichsten Chunks, also diejenigen Abschnitte, die inhaltlich am ehesten zur Frage passen.

In der *Augmentierung* werden die gefundenen Chunks gemeinsam mit der ursprünglichen Frage in einen Prompt eingebettet. Das Sprachmodell erhält so nicht nur die Frage, sondern auch den relevanten Kontext, auf den es seine Antwort stützen soll.

In der abschließenden *Generierung* formuliert das Sprachmodell aus Frage und bereitgestelltem Kontext die Antwort. Da diese auf den übergebenen Chunks beruht, lässt sie sich auf die zugrunde liegenden Quellen zurückführen, was die Nachvollziehbarkeit erhöht und das Risiko von Halluzinationen verringert [1].

Die Qualität der nachfolgenden Phasen hängt unmittelbar von der Indexierung ab. Nur Inhalte, die durch das Chunking sinnvoll voneinander abgegrenzt und vollständig erfasst wurden, können im Retrieval überhaupt gefunden und in der Generierung korrekt wiedergegeben werden.

2.2. Embeddings und Vektor-Retrieval

Um dem Sprachmodell die passenden Textstellen bereitzustellen, muss zunächst bestimmt werden, welche Abschnitte einer Wissensbasis für eine Anfrage relevant sind. Dafür haben sich zwei grundlegende Ansätze etabliert, das term-basierte und das vektor- bzw. embedding-basierte Retrieval [7, S. 338].

Term-basiertes Retrieval

Beim *term-basierten* Retrieval wird die Relevanz auf lexikalischer Ebene bestimmt, also anhand übereinstimmender Schlüsselwörter. Verbreitete Verfahren wie TF-IDF oder BM25 gewichten Terme dabei nach ihrer Häufigkeit im Dokument und ihrer Seltenheit im Gesamtbestand. Der Ansatz ist beim Indexieren wie beim Abfragen sehr schnell, funktioniert ohne großen Anpassungsaufwand und bildet eine starke Ausgangsbasis. Sein Nachteil ergibt sich aus derselben Eigenschaft. Da nur die Wortoberfläche verglichen wird, findet er inhaltlich passende Stellen nicht, wenn diese mit anderen Worten, Synonymen oder Umschreibungen formuliert sind [7, S. 339–341, 345].

Vektorbasiertes Retrieval

Das *vektorbasierte* (embedding-basierte) Retrieval vergleicht demgegenüber nicht die Wörter selbst, sondern deren Bedeutung. Anfrage und Abschnitte werden in numerische Vektoren überführt, sodass inhaltlich verwandte Texte auch bei unterschiedlicher Formulierung als ähnlich erkannt werden. Dieser Ansatz ist robuster gegenüber Synonymen und Paraphrasen und lässt sich durch Optimierung des Embedding-Modells weiter verbessern. Als Nachteil ist er rechnerisch aufwendiger, und die Überführung in Embeddings kann exakte Schlüsselbegriffe wie etwa Referenznummern oder Fachcodes verschleiern, sodass gerade nach solchen präzisen Begriffen schwerer gesucht werden kann [7, S. 341, 345]. In der Praxis werden beide Ansätze daher oft zu einer hybriden Suche kombiniert.

Strukturbasiertes Retrieval

Über die beiden Grundansätze hinaus existieren strukturbasierte Verfahren, die eine explizite Struktur über den Korpus legen, etwa einen Wissensgraphen (GraphRAG) oder einen Baum aus rekursiven Zusammenfassungen (RAPTOR), und so Beziehungen zwischen Textstellen jenseits der reinen Bedeutungsähnlichkeit nutzbar machen [2], [8]. Da sie jedoch die Grenze zwischen Chunking und Retrieval verwischen und das Retrieval in dieser Arbeit konstant gehalten wird, steht hier die bedeutungsbasierte Suche im Vordergrund, weshalb im Folgenden das vektorbasierte Retrieval genauer betrachtet wird.

Embeddings und Ähnlichkeitsmaß

Grundlage des vektorbasierten Retrievals sind *Embeddings*, dichte Vektorrepräsentationen von Text, die dessen Bedeutung so abbilden, dass semantisch ähnliche Textstücke im Vektorraum nahe beieinander liegen [9]. Ein Embedding-Modell überführt dazu sowohl die

indexierten Chunks als auch die Nutzerfrage in Vektoren desselben Raums. Abbildung 1 veranschaulicht diesen Zusammenhang an einfachen Beispielwörtern.

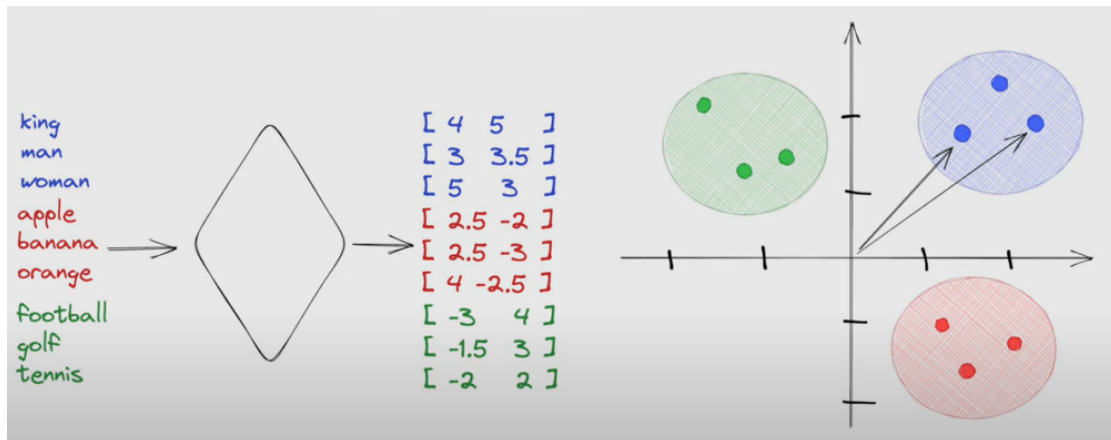


Abbildung 1: Ein Embedding-Modell überführt Wörter in Vektoren, sodass semantisch verwandte Begriffe im Vektorraum nahe beieinander liegen (hier vereinfacht in zwei Dimensionen). Quelle: <https://causewriter.ai/courses/ai-explainers/lessons/vector-embedding/>.

Die Ähnlichkeit zwischen Frage und Chunk wird anschließend über ein Abstandsmaß bestimmt, in der Praxis meist über die Kosinus-Ähnlichkeit, die den Winkel zwischen zwei Vektoren bewertet und damit unabhängig von deren Länge ist. Für eine Anfrage werden die k Chunks mit der höchsten Ähnlichkeit zurückgegeben (Top- k -Suche); große Bestände werden dazu in einer Vektordatenbank abgelegt.

Modellauswahl

Die Auswahl des richtigen Embedding-Modells ist entscheidend für die Qualität des Retrievals [9]. Modelle unterscheiden sich in Architektur, Dimensionalität und unterstützten Sprachen. Um Kandidaten fundiert zu vergleichen, dient das *Massive Text Embedding Benchmark* (MTEB), das Embedding-Modelle standardisiert über verschiedene Aufgaben hinweg bewertet [9]. Werden mehrere Sprachen benötigt, bietet dessen mehrsprachige Erweiterung, das *Massive Multilingual Text Embedding Benchmark* (MMTEB), die bessere Grundlage, da sie Modelle über zahlreiche Sprachen hinweg vergleicht und auch deutschsprachige Retrieval-Aufgaben umfasst [10]. Der hier verwendete EU-GMP-Leitfaden liegt in englischer Sprache vor, während die Anfragen an das System gemischt deutsch- und englischsprachig formuliert sind. Damit deutsche Fragen gegen den englischen Normtext abgeglichen werden können, müssen Anfrage und Chunk trotz unterschiedlicher Sprache im selben Vektorraum liegen; erforderlich ist daher ein mehrsprachiges Modell, das cross-linguales Retrieval beherrscht.

Da ein eigener Modellvergleich nicht Gegenstand dieser Arbeit ist, stützt sich die Auswahl auf diese vorliegenden Benchmarks. Die Wahl fällt auf `multilingual-e5-large`, ein mehrsprachiges Modell, das auf den mehrsprachigen Retrieval-Aufgaben der genannten Benchmarks, für Deutsch und Englisch ebenso wie sprachübergreifend, robust abschneidet [10], [11] und mit rund 560 Mio. Parametern (etwa 1,1 GB als fp16-Gewichte) auch lokal ohne dedizierte GPU handhabbar bleibt. Da das Embedding-Modell über alle Chunking-Strategien hinweg konstant gehalten wird, wirkt es sich nicht als Störgröße auf den Vergleich aus.

2.3. Chunking-Strategien

Die Zerlegung der Dokumente in Chunks bestimmt maßgeblich, welche Textstellen das Retrieval überhaupt finden kann. Ein guter Chunk fasst inhaltlich Zusammengehöriges zusammen und trennt unterschiedliche Themen voneinander. Ist ein Chunk zu groß oder thematisch gemischt, verwässert sein Embedding und erzeugt Rauschen im Retrieval. Der Vektorvergleich zwischen Abfrage und Vergleichsvektoren ist dann nicht mehr genau genug. Wird dagegen ein zusammenhängender Gedanke zerschnitten, erhält das Sprachmodell nur unvollständigen Kontext [4]. Wie sich diese Zerlegung am besten erreichen lässt, ist Gegenstand unterschiedlicher Strategien.

Diese Arbeit knüpft an die Untersuchung von Qu et al. [4] an und greift deren Gedanken auf, den Text entlang seiner Abschnitte zu unterteilen. Die Autoren legen nahe, dass semantisches Chunking bei bereits strukturiertem Text weitgehend von der vorhandenen Gliederung vorgegeben wird. Gerade bei normativen Texten bedeutet das, dass sich einzelne Abschnitte weitgehend isoliert betrachten lassen.

Größenbasiertes Chunking

Am einfachsten arbeitet das *größenbasierte* Chunking, oft als „fixed-size“ bezeichnet. Der Text wird dabei in Abschnitte fester Länge zerlegt, gemessen in Tokens oder Sätzen, meist mit einer Überlappung (*Overlap*) benachbarter Abschnitte, damit an den Schnittstellen kein Kontext verloren geht. Der Ansatz ist schnell, unabhängig vom Dokumentaufbau und dient in Vergleichsstudien regelmäßig als Baseline. Sein Nachteil ist, dass die Schnitte rein nach Länge gesetzt werden und dadurch mitten durch inhaltlich zusammenhängende Passagen verlaufen können.

Semantische Verfahren

Semantische Verfahren setzen die Grenzen stattdessen dort, wo sich der Inhalt ändert. Dazu wird der Text in Sätze zerlegt und jeder Satz eingebettet. Ein *breakpoint-basiertes* Verfahren vergleicht benachbarte Sätze und trennt, sobald deren Embedding-Distanz einen Schwellwert wie etwa ein Perzentil überschreitet, während *clusterbasierte* Verfahren auch nicht benachbarte Sätze zu Chunks gruppieren [4]. Der Mehraufwand solcher Verfahren zahlt sich allerdings nicht zuverlässig aus, wie vergleichende Untersuchungen zeigen [4], [12].

Strukturbasiertes Chunking

Das *strukturbasierte* Chunking nutzt schließlich die im Dokument bereits vorhandene Gliederung. Es trennt entlang von Kapiteln, Abschnitten oder nummerierten Absätzen und kann die jeweilige Überschrift als Kontext mitführen. Bei stark gegliederten Dokumenten wie etwa normativen Texten fallen diese Grenzen häufig mit inhaltlichen Sinneinheiten zusammen, sodass die Struktur unmittelbar als Zerlegungshinweis dient.

Quer zu allen Strategien liegt der Zielkonflikt der *Chunkgröße*. Kleine Chunks von etwa 64 bis 128 Tokens ermöglichen präzises Retrieval für eng umrissene, faktische Fragen, enthalten aber wenig Kontext. Größere Chunks von etwa 512 bis 1024 Tokens liefern mehr Zusammenhang, erhöhen jedoch das Risiko von Rauschen und irrelevanten Passagen. Welche Größe optimal ist, hängt dabei von Aufgabe und Embedding-Modell ab [13].

2.4. Stand der Technik

Im Folgenden wird der aktuelle Forschungsstand zum Chunking und zum Retrieval in regulierten Bereichen zusammengefasst.

Über die in Abschnitt 2.3 genannten Vergleiche von größenbasierten und semantischen Verfahren hinaus zeigen Jimeno Yepes et al. [14] am Beispiel von Finanzberichten, dass auch die Struktur eines Dokuments beim Chunking helfen kann. Statt die Berichte nach Länge zu zerlegen, teilen sie diese nach Strukturelementen wie Überschriften, Fließtext oder Tabellen auf, die zuvor von einem Modell zur Dokumentanalyse erkannt werden, und verbessern damit die Ergebnisse ihres RAG-Systems deutlich. Ein Unterschied zu dieser Arbeit liegt darin, dass die Elemente dort erst durch ein eigenes Modell erkannt werden müssen, während Normen wie der EU-GMP-Leitfaden ihre Struktur über die Absatznummern bereits mitbringen.

Für rechtliche Texte stellen Pipitone und Houir Alami [15] mit LegalBench-RAG einen Benchmark vor, der gezielt nur den Retrieval-Schritt bewertet. Geprüft wird dort, ob

möglichst kleine und genau passende Textstellen gefunden werden, was dem Vorgehen dieser Arbeit mit einzelnen Absätzen als Ground Truth ähnelt.

Einen direkten Vergleich von größenbasiertem und strukturbasiertem Chunking an einem normativen Dokument mit nummerierten Absätzen gibt es nach der Recherche für diese Arbeit bisher nicht, und genau hier setzt die vorliegende Arbeit an.

3. Methodik

3.1. Korpus

Als Korpus dient der EU-GMP-Leitfaden in der englischen Originalfassung, wie ihn die Europäische Kommission als Band 4 der EudraLex-Reihe veröffentlicht [16]. Einbezogen werden Teil I mit seinen neun Kapiteln, die vom pharmazeutischen Qualitätssystem bis zur Selbstinspektion reichen, sowie Annex 1 zur Herstellung steriler Arzneimittel. Die Kapitel des Teils I stammen aus unterschiedlichen Jahren zwischen 2011 und 2014, weshalb ihr jeweiliger Stand dokumentiert wird. Annex 1 liegt in der Fassung vom 25. August 2022 vor und ist seit dem 25. August 2024 vollständig anwendbar.

Bewusst nicht in den Korpus aufgenommen wird Annex 2. Angestrebt ist ein kleiner, sauber annotierbarer Korpus, dessen überschaubarer Umfang eine verlässliche Zuordnung von Fragen zu den belegenden Textstellen erlaubt. Diese Nachvollziehbarkeit wird als methodischer Vorteil gegenüber einer möglichst breiten Abdeckung gewertet.

3.2. Chunking-Strategien im Vergleich

In dieser Arbeit werden zwei Strategien verglichen, die beide mit derselben maximalen Chunkgröße arbeiten und sich nur darin unterscheiden, an welchen Stellen der Text geschnitten wird.

Fixed-Size-Chunking

Für das Fixed-Size-Chunking kommt der `RecursiveCharacterTextSplitter` aus der Bibliothek `LangChain` zum Einsatz. Er zerlegt den Text in Chunks mit höchstens 256 Tokens, wobei sich benachbarte Chunks um 32 Tokens überlappen. Gezählt werden die Tokens mit dem Tokenizer des Embedding-Modells, damit die Chunkgröße genau dem entspricht, was das Modell später verarbeitet. Da das Modell bis zu 512 Tokens verarbeiten kann, liegen 256 Tokens sicher unter dieser Grenze. Der Splitter schneidet zwar bevorzugt an Zeilenumbrüchen, achtet aber nicht darauf, wo ein Absatz beginnt

oder endet, weshalb ein Chunk oft das Ende eines Absatzes, einen oder mehrere ganze Absätze und den Anfang des nächsten enthält.

Struktur-Split

Beim Struktur-Split wird der Text dagegen zuerst an den Absatznummern getrennt. Dafür sucht ein regulärer Ausdruck nach Zeilen, die mit einer Nummer wie 4.1 oder 4.2 beginnen, sodass jeder nummerierte Absatz zu einem eigenen Chunk wird. Der Text vor dem ersten nummerierten Absatz, also der Abschnitt „Principle“ in Teil I und „Scope“ in Annex 1, wird als eigener Abschnitt behandelt. Ist ein Absatz länger als 256 Tokens, wird er mit demselben Splitter und denselben Einstellungen wie beim Fixed-Size-Chunking weiter unterteilt, wobei alle Teile die Nummer ihres Absatzes behalten. Das betrifft 41 von 577 Abschnitten.

Kleine Absätze wurden bewusst nicht zusammengelegt. Der Gedanke dahinter ist, dass der Absatz die natürliche Einheit einer Norm ist und ein Chunk mit nur einem Absatz auch nur ein Thema behandelt. Dadurch sind die Struktur-Chunks im Durchschnitt allerdings deutlich kleiner. Im Mittel haben sie 97 Tokens, die Fixed-Size-Chunks dagegen 209 Tokens. Da die Chunkgröße das Retrieval selbst beeinflusst [13], muss dieser Unterschied bei der Auswertung beachtet werden. Er wird jedoch in Kauf genommen, da er keine zweite Stellgröße ist, sondern eine direkte Folge davon, entlang der Absätze zu schneiden. Wie er in der Auswertung berücksichtigt wird, beschreibt Abschnitt 3.4.

Damit der Struktur-Split die Absätze richtig erkennt, wird der Text vorher an drei Stellen bereinigt. Das Glossar am Ende von Annex 1 wird entfernt, da es keine Anforderungen enthält und sonst am letzten Absatz hängen würde. Da dieser Schritt vor dem Chunking passiert, arbeiten beide Strategien mit demselben Text. Außerdem werden Zwischenüberschriften dem Absatz zugeordnet, den sie einleiten, und Chunks verworfen, die nur aus einer Seitenzahl bestehen. Wie diese Schritte technisch umgesetzt sind, beschreibt Kapitel 4.

3.3. Fragenset und Ground Truth

Das Fragenset umfasst 55 Fragen, die mit einem KI-Sprachmodell erzeugt und anschließend von Hand geprüft wurden. Dafür kommt für alle Dokumente derselbe Prompt zum Einsatz, an den die nummerierten Absätze eines Dokuments angehängt werden. Jeder Absatz steht dabei mit seiner Nummer in eckigen Klammern, zum Beispiel [4.32], damit das Modell seine Fragen eindeutig einem Absatz zuordnen kann.

Der Prompt gibt dem Modell mehrere Regeln vor. So muss sich jede Frage aus genau einem Absatz beantworten lassen, und die Nummer dieses Absatzes muss genau so

angegeben werden, wie sie im Text steht. Formuliert werden sollen die Fragen so, wie eine Fachperson aus dem GMP-Umfeld sie stellen würde, wobei Ja-Nein-Fragen, Fragen zum Aufbau des Dokuments und Fragen zu Deckblatt oder Geltungsbereich ausgeschlossen sind. Außerdem soll jede Frage mit anderen Worten formuliert sein, statt Sätze aus dem Absatz zu übernehmen. Der Gedanke dahinter ist, dass das Retrieval sonst nur gleiche Wörter findet und nicht gleiche Bedeutung. Etwa die Hälfte der Fragen soll auf Deutsch und die andere Hälfte auf Englisch gestellt werden. Die Antwort des Modells folgt einem festen JSON-Format, das zu jeder Frage die Sprache, das Dokument und die Nummer des richtigen Absatzes enthält. So lassen sich die Fragen direkt einlesen und automatisch darauf prüfen, ob die angegebenen Absatznummern im Dokument tatsächlich vorkommen.

Die Anzahl der Fragen je Dokument richtet sich nach seiner Anzahl an Absätzen, sodass Annex 1 mit etwa der Hälfte aller Absätze 27 der 55 Fragen erhält und jedes Kapitel aus Teil I mindestens zwei. Da Annex 1 für eine einzige Anfrage zu lang ist, wird er dafür nach seinen Hauptabschnitten aufgeteilt. Von den Fragen sind 29 auf Deutsch und 26 auf Englisch formuliert, sodass deutsche Fragen dem englischen Text zugeordnet werden müssen (siehe Abschnitt 2.2). Da viele Absätze in Annex 1 Fristen und Grenzwerte festlegen, handelt es sich überwiegend um Faktenfragen.

Als *Ground Truth* dient der nummerierte Absatz im Originaldokument, der unabhängig von der Chunking-Strategie ist. Ein Chunk gilt als Treffer, wenn er diesen Absatz enthält. Beim Struktur-Split lässt sich das an der Absatznummer ablesen, bei den Fixed-Size-Chunks zählt ein Absatz dazu, wenn sich beide auf mindestens der Hälfte der kürzeren Textstelle überschneiden. Da Absatznummern wie 2.2 in mehreren Dokumenten vorkommen, wird immer das Paar aus Dokument und Absatznummer verglichen.

3.4. Evaluation

Für jede Frage werden die Chunks beider Strategien nach ihrer Ähnlichkeit zur Frage sortiert. Betrachtet werden jeweils die ersten k Chunks dieser Rangliste, also genau die Chunks, die ein RAG-System dem Sprachmodell als Kontext übergeben würde.

Als wichtigste Kennzahl dient der *Recall@k*. Er gibt an, bei welchem Anteil der Fragen der richtige Absatz unter den ersten k Chunks ist. Ein *Recall@1* von 0,8 heißt also, dass bei 80 % der Fragen der richtige Absatz auf dem ersten Platz steht. Da es zu jeder Frage genau einen richtigen Absatz gibt, ist der Recall einer einzelnen Frage entweder 0 oder 1. Ausgewertet werden die Werte $k = 1, 3, 5$ und 10. Zusätzlich wird k so weit erhöht, bis beide Strategien für jede Frage den richtigen Absatz gefunden haben, denn im regulierten Umfeld darf keine relevante Stelle fehlen. Dieses kleinste vollständige k zeigt, wie viel Kontext eine Strategie dafür braucht.

Außerdem wird der *Mean Reciprocal Rank* (MRR) berechnet. Dafür wird für jede Frage der Kehrwert des Platzes genommen, auf dem der erste Treffer steht, und über alle Fragen gemittelt. Ein Treffer auf Platz 1 zählt also 1, auf Platz 2 nur noch 0,5, sodass der MRR zeigt, wie weit oben der richtige Absatz im Durchschnitt landet.

Zu jedem k wird zudem angegeben, wie viele Tokens die ersten k Chunks im Mittel zusammen enthalten. Das ist wichtig, weil die Fixed-Size-Chunks etwa doppelt so groß sind wie die Struktur-Chunks und bei gleichem k deshalb deutlich mehr Text liefern. Damit haben sie auch mehr Gelegenheit, den richtigen Absatz dabeizuhaben, und der Vergleich bei gleichem k bevorzugt das Fixed-Size-Chunking. Schneidet der Struktur-Split trotzdem besser ab, ist das ein vorsichtiges und damit belastbares Ergebnis.

Ob ein Unterschied zwischen den Strategien auch zufällig entstanden sein könnte, wird mit einem paarweisen Test geprüft, bei dem für jede Frage beide Strategien direkt miteinander verglichen werden. Da der Recall pro Frage nur 0 oder 1 sein kann, sind dabei nur die Fragen interessant, bei denen eine Strategie trifft und die andere nicht. Für diese Fragen wurde sich für einen exakten, zweiseitigen Vorzeichentest entschieden, der prüft, ob eine Strategie deutlich häufiger gewinnt, als es bei reinem Zufall zu erwarten wäre. Um zu prüfen, ob sich die Fixed-Size-Chunks auf Rang 1 und dahinter im Anteil des richtigen Absatzes unterscheiden, kommt außerdem ein Mann-Whitney-U-Test zum Einsatz (siehe Abschnitt 5.2). Er wird einseitig durchgeführt, da die vermutete Ursache eine bestimmte Richtung vorgibt. Als Signifikanzniveau werden jeweils 5 % verwendet.

Die Qualität der erzeugten Antworten wird in dieser Arbeit nicht bewertet. Das Retrieval ist die Voraussetzung für eine gute Antwort, denn ein Absatz, der nicht im Kontext steht, kann vom Sprachmodell auch nicht genutzt werden. Hinzu kommt, dass automatische Bewertungsverfahren für Antworten noch unzuverlässig sind. So konnte die Kennzahl *Faithfulness* des Werkzeugs RAGAS in einer vergleichbaren Studie in 44 % der Fälle gar nicht berechnet werden, weil die Bewertung abbrach oder ungültige Werte lieferte [12, S. 3]. Aus diesen Gründen konzentriert sich die Arbeit auf das Retrieval.

4. Implementierung

In diesem Kapitel wird beschrieben, wie der Versuchsaufbau technisch umgesetzt ist. Die gesamte Pipeline ist in Python als Jupyter-Notebook geschrieben, das zusammen mit dem Korpus und den Fragen im Repository https://github.com/tommi-j/Hausarbeit_NLP_MatNr_30348877 abgelegt ist.

Beide Strategien durchlaufen dieselbe Pipeline. Zuerst werden die PDF-Dateien eingelesen, dann wird der Text in Chunks zerlegt und jeder Chunk in einen Vektor umgewandelt, bevor schließlich für jede Frage nach den ähnlichsten Chunks gesucht wird. Nur der

Schritt des Chunkings unterscheidet sich zwischen den beiden Strategien, alle anderen Schritte sind identisch. Unterschiede in den Ergebnissen gehen deshalb allein auf das Chunking zurück.

Für das Auslesen der PDF-Dateien kommt PyMuPDF zum Einsatz, für die Zerlegung der Text-Splitter aus LangChain. Die Embeddings werden mit der Bibliothek **sentence-transformers** und dem Modell **multilingual-e5-large** erzeugt. Das Modell erwartet vor jedem Text einen kurzen Präfix, der angibt, ob es sich um eine Frage oder um einen Dokumentabschnitt handelt. Fragen beginnen deshalb mit **query:** und Chunks mit **passage:**, da das Modell mit diesen Präfixen trainiert ist [11] und sie deshalb auch bei der Anwendung gesetzt werden müssen.

Alle Vektoren werden auf die Länge 1 normiert, sodass die Kosinus-Ähnlichkeit zwischen Frage und Chunk einfach dem Skalarprodukt der beiden Vektoren entspricht. Mit rund 1 000 Chunks ist der Korpus so klein, dass sich alle Ähnlichkeiten auf einmal mit NumPy berechnen lassen. Eine Vektordatenbank, wie sie in Abschnitt 2.2 beschrieben ist, ist deshalb nicht nötig. Außerdem ist die Suche dadurch exakt, da keine Näherungsverfahren zum Einsatz kommen.

Beim Auslesen mit PyMuPDF geht die Formatierung wie Schriftgröße oder Fettdruck verloren, sodass Überschriften, Seitenzahlen und angehängte Teile wie das Glossar danach als normale Zeilen im Text stehen. Das Glossar wird deshalb direkt beim Einlesen an seiner Überschrift abgeschnitten.

Den Kern des Struktur-Splits bildet eine Funktion, die für jeden Absatz Anfang und Ende im Text bestimmt. Listing 1 zeigt eine gekürzte Fassung davon. Der reguläre Ausdruck findet zunächst alle Zeilen, die mit einer Absatznummer beginnen, wobei jeder Absatz bis zum Beginn des nächsten reicht. Da eine Zwischenüberschrift im ausgelesenen Text direkt vor der Nummer des nächsten Absatzes steht, landet sie dabei zunächst am Ende des vorherigen Absatzes. Deshalb wird anschließend geprüft, ob die letzte Zeile eines Absatzes eine Überschrift ist, die dann an den Anfang des folgenden Absatzes verschoben wird. Als Überschrift gilt eine Zeile, die kein Satzzeichen am Ende hat, höchstens 15 Wörter lang ist und mit einem Großbuchstaben oder mit der Nummer des nächsten Abschnitts beginnt. Da mehrere Überschriften direkt übereinander stehen können, etwa eine Kapitel- und eine Unterüberschrift, wird diese Prüfung so lange wiederholt, bis keine weitere Überschrift mehr gefunden wird.

Nach dem Chunking werden beim Struktur-Split außerdem alle Chunks mit weniger als fünf Tokens verworfen. Beim Auslesen landen einzelne Seitenzahlen in eigenen Zeilen und bleiben als Chunks aus nur einem Token übrig, insgesamt acht Stück. Da der kleinste Chunk mit echtem Inhalt 15 Tokens hat, liegt die Grenze von fünf Tokens dazwischen, sodass nur die Seitenzahlen entfernt werden und kein Inhalt verloren geht.

Listing 1: Kern des Struktur-Splits (gekürzt, ohne Vorspann).

```
absatz_muster = re.compile(r"(?m)^\s*(\d+\.\d+)\s")

def absatz_spans(text):
    treffer = list(absatz_muster.finditer(text))
    enden = [m.start() for m in treffer[1:]] + [len(text)]
    spans = [[m.group(1), m.start(), e] for m, e in zip(treffer, enden)]
    for vorher, nachher in zip(spans, spans[1:]):
        while (s := ueberschrift_start(
            text, *vorher[1:], nachher[0])) is not None:
            vorher[2] = nachher[1] = s
    return spans

def struktur_split(text):
    return [(nr, text[a:b].strip()) for nr, a, b in absatz_spans(text)]
```

Die Positionen der Absätze werden auch für die Fixed-Size-Chunks genutzt. Für jeden Fixed-Size-Chunk wird dazu seine Position im Text gesucht und mit den Absatzgrenzen verglichen. Auf diese Weise verwenden beide Strategien genau dieselben Absatzgrenzen, wenn geprüft wird, ob ein Chunk den richtigen Absatz enthält.

Die Ergebnisse lassen sich vollständig wiederholen, da das Chunking und die Suche keine Zufallselemente enthalten und jeder Durchlauf deshalb dieselben Zahlen liefert. Nur die Fragen stammen von einem Sprachmodell und würden bei einer neuen Erzeugung anders ausfallen. Aus diesem Grund sind die fertigen Fragen zusammen mit dem Notebook gespeichert. Verwendet wurden Python 3.11, PyMuPDF 1.28, `langchain-text-splitters` 1.1, `sentence-transformers` 6.1, `transformers` 5.17 und NumPy 2.2.

5. Ergebnisse und Diskussion

In diesem Kapitel werden die Ergebnisse des Vergleichs vorgestellt und anschließend diskutiert.

5.1. Retrieval-Ergebnisse

Tabelle 1 fasst die Ergebnisse beider Strategien zusammen. Der deutlichste Unterschied zeigt sich auf dem ersten Platz, denn der Struktur-Split findet bei 85,5% der Fragen den richtigen Absatz auf Rang 1, das Fixed-Size-Chunking nur bei 69,1%. Das sind neun Fragen mehr, bei denen der Struktur-Split sofort die richtige Stelle liefert, und das bei kleinerem Kontext. Der erste Struktur-Chunk hat im Mittel nur 126 Tokens, der erste

Strategie	R@1	R@3	R@5	R@10	R@13	MRR
Fixed-Size	0,691	0,891	0,927	1,000	1,000	0,804
Ø Tokens	226	693	1 159	2 265	2 936	
Struktur-Split	0,855	0,909	0,964	0,982	1,000	0,898
Ø Tokens	126	407	689	1 377	1 867	

Tabelle 1: Recall@ k und MRR beider Strategien für $n = 55$ Fragen. Die Zeilen „Ø Tokens“ geben an, wie viele Tokens die ersten k Chunks im Mittel zusammen enthalten.

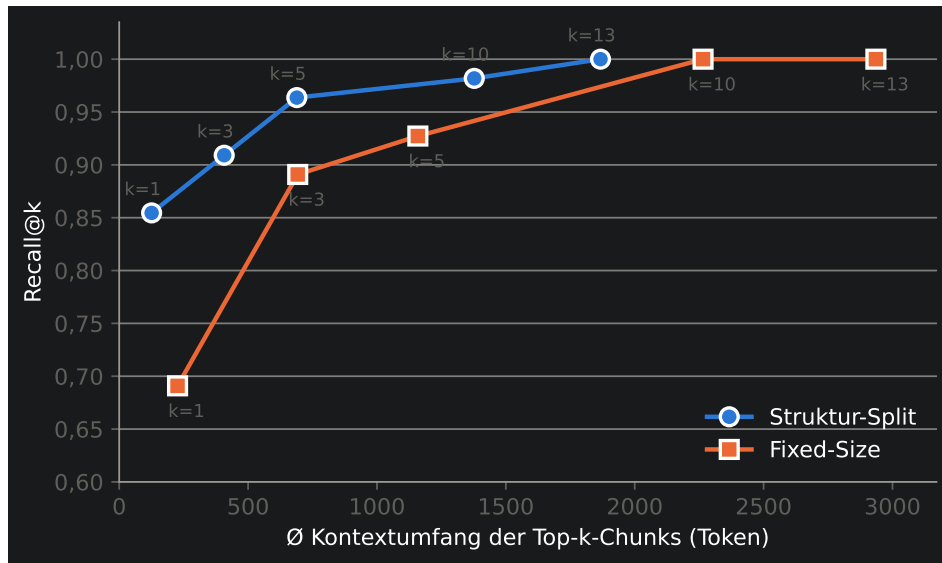


Abbildung 2: Recall@ k über dem mittleren Textumfang der ersten k Chunks für $k = 1, 3, 5, 10$ und 13 . Die y-Achse beginnt bei 0,6.

Fixed-Size-Chunk dagegen 226 Tokens. Auch der MRR liegt mit 0,898 gegenüber 0,804 höher, der richtige Absatz landet beim Struktur-Split also im Durchschnitt weiter oben.

Betrachtet man die Fragen einzeln, gibt es elf Fragen, bei denen der Struktur-Split auf Rang 1 trifft und das Fixed-Size-Chunking danebenliegt. Umgekehrt ist das nur bei zwei Fragen der Fall. Der exakte Vorzeichentest ergibt dafür $p = 0,022$, womit der Unterschied auf Rang 1 auf dem 5%-Niveau signifikant ist.

Mit steigendem k wird der Abstand kleiner. Bei $k = 3$ und $k = 5$ liegt der Struktur-Split zwar weiterhin vorne, der Unterschied ist aber nicht mehr signifikant. Bei $k = 10$ findet das Fixed-Size-Chunking sogar für alle 55 Fragen den richtigen Absatz und der Struktur-Split nur für 54, wofür das Fixed-Size-Chunking allerdings 2265 Tokens braucht und der Struktur-Split nur 1377. Ab $k = 13$ deckt auch der Struktur-Split alle 55 Fragen ab, und zwar mit 1867 Tokens, also immer noch mit weniger Text, als das Fixed-Size-Chunking bei $k = 10$ benötigt.

Abbildung 2 stellt diesen Zusammenhang grafisch dar. Auf der x-Achse ist dabei nicht k aufgetragen, sondern die Menge an Text, die bei dem jeweiligen k zusammenkommt. Da die Linie des Struktur-Splits durchgehend links oberhalb der Linie des Fixed-Size-Chunkings liegt, findet der Struktur-Split bei gleicher Textmenge immer mindestens so oft den richtigen Absatz.

5.2. Diskussion

Die Ergebnisse stützen die Hypothese H1 vor allem für den ersten Platz. Dort findet der Struktur-Split signifikant häufiger den richtigen Absatz, obwohl der Vergleich bei gleichem k das Fixed-Size-Chunking bevorzugt. Für größere k lässt sich H0 dagegen nicht verwerfen. Der Vorteil des Struktur-Splits liegt also vor allem dort, wo nur wenig Kontext an das Sprachmodell übergeben werden soll oder kann.

Mögliche Ursache

Eine mögliche Erklärung ist die in Abschnitt 2.3 beschriebene Verwässerung. Da ein Fixed-Size-Chunk oft mehrere Absätze zu unterschiedlichen Themen enthält, passt sein Embedding nicht mehr eindeutig zu einer Frage. Einen Hinweis darauf gibt der Anteil, den der richtige Absatz am Text des jeweiligen Fixed-Size-Chunks hat. Steht der Chunk auf Rang 1, liegt dieser Anteil im Mittel bei 49 %, steht er weiter hinten, nur bei 37 %. Der Unterschied geht damit zwar in die erwartete Richtung, ist mit $p = 0,066$ im einseitigen Mann-Whitney-U-Test aber nicht signifikant. Außerdem gibt es Gegenbeispiele wie die Frage zu Absatz 2.17, bei der ein Fixed-Size-Chunk aus fünf Absätzen trotzdem auf Rang 1

steht, obwohl nur 18 % seines Textes zum richtigen Absatz gehören. Die Verwässerung ist damit eine plausible, aber keine vollständige Erklärung.

Fehleranalyse

Ein Blick auf die einzelnen Fragen zeigt, wogegen die Strategien verlieren. Beim Fixed-Size-Chunking steht meist ein Chunk mit einem thematisch nahen, aber falschen Absatz vorne, häufig ein direkter Nachbar aus demselben Abschnitt. So landet bei der Frage zu Absatz 8.22 in Annex 1 ein Chunk mit den Absätzen 8.23 und 8.24 auf Rang 1. Vor solchen Verwechslungen ist aber auch der Struktur-Split nicht geschützt. Bei der Frage zur Identitätsprüfung von Ausgangsstoffen (Kapitel 5, Absatz 5.35) steht der Nachbarabsatz 5.36 vorne, der ebenfalls die Prüfung von Ausgangsstoffen behandelt, und der richtige Absatz folgt erst auf Rang 13. Da beim Struktur-Split jeder Chunk nur einen Absatz enthält, konkurrieren thematisch ähnliche Nachbarn hier direkt miteinander.

Einordnung

Für die Vermutung von Qu et al. [4] liefern die Ergebnisse damit eine erste Bestätigung an einem echten Normdokument, da das Trennen entlang der Struktur die richtige Stelle häufiger nach vorne bringt. Deutlich ist der Effekt allerdings nur auf dem ersten Platz, was zu deren Befund passt, dass sich Chunking-Verfahren in ihrer Wirkung meist nur wenig unterscheiden und ihr Einfluss oft hinter dem des Embedding-Modells zurücktritt [4, S. 4].

Für den Einsatz im regulierten Umfeld ist neben dem ersten Treffer vor allem wichtig, dass keine relevante Stelle fehlt. Auf den ersten Blick scheint hier das Fixed-Size-Chunking im Vorteil zu sein, weil es schon bei $k = 10$ alle Fragen abdeckt. Berücksichtigt man aber die Textmenge, dreht sich das Bild, denn der Struktur-Split deckt alle Fragen mit 1 867 Tokens ab, während das Fixed-Size-Chunking dafür 2 265 Tokens braucht. Für die Vollständigkeit zählt also nicht die Anzahl der Chunks, sondern wie viel Text dem Sprachmodell übergeben werden kann.

Schwierigkeiten und Erkenntnisse

Bei der Umsetzung sind einige Schwierigkeiten aufgetreten, die vor allem die Aufbereitung der Daten betrafen. Beim Auslesen von PDF-Dateien geht die Formatierung verloren, sodass die Gliederung eines Dokuments nicht mehr direkt erkennbar ist und aus dem reinen Text wiederhergestellt werden muss, was fehleranfällig ist. Einige dieser Fehler sind erst bei der Kontrolle der Fragen aufgefallen und mussten nachträglich behoben

werden (siehe Kapitel 4). Da der Struktur-Split auf die Gliederung angewiesen ist, wirken sich solche Fehler vor allem auf ihn aus.

Auch die Auswertung und das Fragenset haben mehr Sorgfalt erfordert als zunächst angenommen. Damit Treffer richtig bewertet werden können, muss jeder Absatz eindeutig zugeordnet sein, auch über mehrere Dokumente hinweg. Beim Fragenset hat sich außerdem gezeigt, dass eine gleiche Anzahl an Fragen je Dokument das Ergebnis verzerrt, weil die Dokumente sehr unterschiedlich lang sind.

Aus diesen Erfahrungen lassen sich einige Erkenntnisse über das Problem selbst ableiten. Die eigentliche Schwierigkeit bei normativen Texten liegt weniger darin, das richtige Thema zu finden, als darin, ähnliche Anforderungen voneinander zu unterscheiden, denn beide Strategien scheitern vor allem an thematisch nahen Absätzen. Hinzu kommt, dass das Ergebnis stark von der Datenaufbereitung abhängt und schon kleine Fehler beim Erkennen der Struktur die Ergebnisse spürbar verändern. Nicht zuletzt ist die Anzahl der Chunks allein kein fairer Maßstab, wenn die Chunks unterschiedlich groß sind. Erst der Blick auf die Textmenge zeigt, welche Strategie wirklich effizienter ist.

Einschränkungen

Bei der Bewertung der Ergebnisse sind einige Einschränkungen zu beachten. Mit 55 Fragen ist die Stichprobe klein, sodass kleinere Unterschiede statistisch nicht sicher nachgewiesen werden können. Außerdem beschränkt sich die Untersuchung auf einen Korpus und ein Embedding-Modell. Die Fragen stammen von einem Sprachmodell und sind überwiegend Faktenfragen zu einem einzelnen Absatz, während Anforderungen, die über mehrere Absätze verteilt sind, im Fragenset nicht vorkommen, obwohl gerade dort die Vollständigkeit entscheidend wäre. Seitenzahlen und Fußnoten, die mitten in Absätzen stehen, bleiben im Text und können die Embeddings leicht verfälschen. Schließlich sind die Chunks beider Strategien unterschiedlich groß, was die Angabe der Textmenge zwar sichtbar macht, aber nicht vollständig ausgleicht.

6. Fazit und Ausblick

Ziel der Arbeit war es herauszufinden, ob ein Struktur-Split entlang der nummerierten Absätze die Retrieval-Qualität eines RAG-Systems für den EU-GMP-Leitfaden gegenüber einem Fixed-Size-Chunking verbessert. Dazu wurden beide Strategien mit 55 Fragen verglichen, während das übrige System unverändert blieb.

Die Ergebnisse zeigen, dass der Struktur-Split den richtigen Absatz deutlich häufiger auf den ersten Platz bringt. Mit einem Recall@1 von 0,855 gegenüber 0,691 ist der Unterschied signifikant, und das bei nur etwa halb so viel Text. Werden mehr Chunks betrachtet, gleichen sich beide Strategien an, wobei der Struktur-Split für eine vollständige Abdeckung aller Fragen zwar mehr Chunks, aber weniger Text benötigt als das Fixed-Size-Chunking. Die Hypothese H1 wird damit für den ersten Platz gestützt, für größere k dagegen nicht.

Für die Praxis lässt sich daraus eine einfache Empfehlung ableiten. Liegen Dokumente mit einer klaren Nummerierung vor, wie es bei Normen und Richtlinien üblich ist, lohnt es sich, diese Struktur beim Chunking zu nutzen. Der Aufwand ist gering, da im Wesentlichen ein regulärer Ausdruck über die Absatznummern ausreicht, und der Vorteil zeigt sich gerade dann, wenn nur wenig Kontext an das Sprachmodell übergeben werden kann.

Für die Zukunft wäre es sinnvoll, als Nächstes die erzeugten Antworten zu untersuchen und dabei beiden Strategien die gleiche Textmenge zur Verfügung zu stellen. So würde sich zeigen, ob der Vorteil beim Retrieval auch zu besseren Antworten führt. Ebenso könnte ein semantisches Chunking im selben Aufbau getestet werden, um den Vergleich von Qu et al. [4] an einem echten Normdokument zu vervollständigen. Interessant wären außerdem Fragen, deren Antwort über mehrere Absätze verteilt ist, sowie weitere Korpora wie die übrigen Annexe des EU-GMP-Leitfadens oder andere Normen. Für größere Korpora käme zudem eine hybride Suche aus Schlüsselwörtern und Embeddings infrage.

Literatur

- [1] P. Lewis, E. Perez, A. Piktus u. a., „Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,“ in *Advances in Neural Information Processing Systems*, Bd. 33, Curran Associates, Inc., 2020, S. 9459–9474. Adresse: <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html> (besucht am 30.08.2026).
- [2] Y. Gao, Y. Xiong, X. Gao u. a. „Retrieval-Augmented Generation for Large Language Models: A Survey.“ arXiv: 2312.10997 [cs.CL]. (27. März 2024), Adresse: <http://arxiv.org/abs/2312.10997> (besucht am 14.08.2026), Vorveröffentlichung.
- [3] I. Ni'Mah, L. R. Aini, R. Fajri, S. Pebiana, N. N. Hidayati und R. Wijayanti, „Evaluating Retrieval Augmented Generation (RAG) Chunking Strategy for Question Answering in Indonesian Law of The Sea,“ in *2025 International Conference on Computer, Control, Informatics and Its Applications (IC3INA)*, Okt. 2025, S. 190–195. DOI: 10.1109/IC3INA68387.2025.11325153. Adresse: <https://ieeexplore.ieee.org/document/11325153> (besucht am 27.09.2026).
- [4] R. Qu, R. Tu und F. Bao. „Is Semantic Chunking Worth the Computational Cost?“ arXiv: 2410.13070 [cs.CL]. (16. Okt. 2024), Adresse: <http://arxiv.org/abs/2410.13070> (besucht am 28.09.2026), Vorveröffentlichung.
- [5] B. e.V. „Internet-Suche im Wandel: Die Hälfte nutzt bereits KI-Chats | Presseinformation | Bitkom e. V.“ (21. Nov. 2025), Adresse: <https://www.bitkom.org/Presse/Presseinformation/Internet-Suche-Wandel-Haelfte-nutzt-KI-Chats> (besucht am 27.09.2026).
- [6] M. Lu, C. Li, P. Wang u. a., „Loki’s Dance of Illusions: A Comprehensive Survey of Hallucination in Large Language Models,“ *IEEE Transactions on Computational Social Systems*, S. 1–32, 2026, ISSN: 2329-924X. DOI: 10.1109/TCSS.2026.3661295. Adresse: <https://ieeexplore.ieee.org/document/11456255> (besucht am 27.09.2026).
- [7] *Generative KI-Systeme Entwickeln - 11772551*. Adresse: https://content-select-com.ezproxy.fhb.fh-swf.de/media/moz_viewer/5216200b-9cce-471c-91a9-c814138602c6/language:de (besucht am 23.09.2026).
- [8] P. Sarthi, S. Abdullah, A. Tuli, S. Khanna, A. Goldie und C. D. Manning. „RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval.“ arXiv: 2401.18059 [cs.CL]. (31. Jan. 2024), Adresse: <http://arxiv.org/abs/2401.18059> (besucht am 28.09.2026), Vorveröffentlichung.

- [9] J. O. Abimbola, G. S. Kuaban und S. A. Ajayi, „Open-Source Embedding Models: A Comprehensive Survey of Techniques, Benchmarks, and Applications,“ *IEEE Access*, Jg. 14, S. 41 284–41 302, 2026, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2026.3670100. Adresse: <https://ieeexplore.ieee.org/document/11426790> (besucht am 27.09.2026).
- [10] K. Enevoldsen, I. Chung, I. Kerboua u. a. „MMTEB: Massive Multilingual Text Embedding Benchmark.“ arXiv: 2502.13595 [cs.CL]. (13. Nov. 2025), Adresse: <http://arxiv.org/abs/2502.13595> (besucht am 27.09.2026), Vorveröffentlichung.
- [11] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder und F. Wei. „Multilingual E5 Text Embeddings: A Technical Report.“ arXiv: 2402.05672 [cs.CL]. (8. Feb. 2024), Adresse: <http://arxiv.org/abs/2402.05672> (besucht am 28.09.2026), Vorveröffentlichung.
- [12] V. J. J. Kreileder, J. Reisinger und A. Fischer. „Evaluating Chunking Strategies for Retrieval-Augmented Generation on Academic Texts.“ arXiv: 2607.01852 [cs.IR]. (2. Juli 2026), Adresse: <http://arxiv.org/abs/2607.01852> (besucht am 28.09.2026), Vorveröffentlichung.
- [13] S. R. Bhat, M. Rudat, J. Spiekermann und N. Flores-Herr. „Rethinking Chunk Size For Long-Document Retrieval: A Multi-Dataset Analysis.“ arXiv: 2505.21700 [cs.IR]. (29. Mai 2025), Adresse: <http://arxiv.org/abs/2505.21700> (besucht am 28.09.2026), Vorveröffentlichung.
- [14] A. J. Yepes, Y. You, J. Milczek, S. Laverde und R. Li. „Financial Report Chunking for Effective Retrieval Augmented Generation.“ arXiv: 2402.05131 [cs.CL]. (16. März 2024), Adresse: <http://arxiv.org/abs/2402.05131> (besucht am 30.09.2026), Vorveröffentlichung.
- [15] N. Pipitone und G. H. Alami. „LegalBench-RAG: A Benchmark for Retrieval-Augmented Generation in the Legal Domain.“ arXiv: 2408.10343 [cs.AI]. (19. Aug. 2024), Adresse: <http://arxiv.org/abs/2408.10343> (besucht am 30.09.2026), Vorveröffentlichung.
- [16] „EudraLex - Volume 4 - Public Health - European Commission.“ (22. Sep. 2026), Adresse: https://health.ec.europa.eu/medicinal-products/eudralex/eudralex-volume-4_en (besucht am 30.09.2026).

Erklärung

Ich erkläre hiermit, dass ich die vorliegende Arbeit selbstständig verfasst und dabei keine anderen als die angegebenen Hilfsmittel benutzt habe. Sämtliche Stellen der Arbeit, die im Wortlaut oder dem Sinn nach Werken anderer Autoren entnommen sind, habe ich als solche kenntlich gemacht. Die Arbeit wurde bisher weder gesamt noch in Teilen einer anderen Prüfungsbehörde vorgelegt und auch noch nicht veröffentlicht.

30. September 2026

Tommi Jlassi